Ex:1A	Text Generation using N-gram Language model

Learning Objective:
Implement an N-Gram–based predictive text model using bigram and trigram probability distributions. This helps to understand how contextual probability influences next-word prediction.

Algorithm:

1. Initialize the environment by installing and importing the Natural Language Toolkit (nltk). Download the **Brown Corpus** (for text data) and the **Universal Tagset** (for simplified Part-of-Speech tags).
2. Extract raw sentences from the Brown Corpus using brown.sents().
3. **Tokenization:** Convert all words to lowercase and flatten the nested list into a single list of tokens to verify the total volume of training data.
4. Build an N-Gram Model for Bigram and Trigram.
5. Predict Next Word and display Top-5 Suggestions
6. Test Bigram & Trigram Prediction on certain words.

Program:

```
import nltk
from nltk.corpus import brown
from nltk.util import ngrams
from collections import defaultdict, Counter
import string

nltk.download('brown')

sentences = brown.sents()

tokens = []
for sent in sentences:
    for word in sent:
        word = word.lower()
        if word not in string.punctuation:
            tokens.append(word)

print("\n=========================================")
print(" N-Gram Language Model ")
print("=========================================")
```

```
print(f"Total Tokens in Corpus : {len(tokens)}")

def build_ngram_model(tokens, n):
    model = defaultdict(Counter)
    for gram in ngrams(tokens, n):
        context = gram[:-1]     # first (n-1) words
        target = gram[-1]       # nth word
        model[context][target] += 1
    return model

def predict_next_words(model, context, top_n=5):
    if context in model:
        return model[context].most_common(top_n)
    else:
        return []

n = int(input("\nEnter the value of N (e.g., 2 for Bigram, 3 for Trigram): "))

if n < 2:
    print("N must be >= 2")
    exit()

model = build_ngram_model(tokens, n)

context_input = input(f"Enter {n-1} word(s) separated by space: ").lower().split()
context = tuple(context_input)

predictions = predict_next_words(model, context)

print("\n-----------------------------------------")
print(f" N-Gram Model : {n}-Gram")
print(f" Context     : {' '.join(context)}")
print("-----------------------------------------")

if predictions:
    print(" Top-5 Next Word Predictions:")
    print("-----------------------------------------")
    for i, (word, freq) in enumerate(predictions, start=1):
        print(f" {i}. {word:<10} | Frequency : {freq}")
else:
    print(" No predictions found for the given input.")

print("-----------------------------------------")
```

Output:

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!

========================================
 N-Gram Language Model
========================================
Total Tokens in Corpus : 1034378

Enter the value of N (e.g., 2 for Bigram, 3 for Trigram): 3
Enter 2 word(s) separated by space: the world

----------------------------------------
 N-Gram Model : 3-Gram
 Context      : the world
----------------------------------------
 Top-5 Next Word Predictions:
----------------------------------------
 1. of         | Frequency : 21
 2. and        | Frequency : 19
 3. the        | Frequency : 17
 4. in         | Frequency : 17
 5. is         | Frequency : 14
----------------------------------------
```

Learning Outcome:

Upon completion of this experiment, the bigram and trigram probability distributions for next-word prediction was implemented and evaluated using the Brown Corpus.

Ex:1B	Hidden Markov Model (HMM) based Predictive Text System

Learning Objective:
To implement a predictive text system using Hidden Markov Model (HMM), enabling students to understand contextual probability, POS transitions, and next-word prediction using the Brown Corpus.

Algorithm:

1. Initialize the environment by installing and importing the Natural Language Toolkit (nltk). Download the Brown Corpus (for text data) and the Universal Tagset (for simplified Part-of-Speech tags).
2. Extract raw sentences from the Brown Corpus using brown.sents().
3. Tokenization: Convert all words to lowercase and flatten the nested list into a single list of tokens to verify the total volume of training data.
4. Load the tagged version of the corpus (brown.tagged_sents) which provides the "Hidden States" (POS tags) linked to the "Observations" (words).
5. Train the HMM Model. Use the HiddenMarkovModelTrainer to perform Supervised Learning. Calculate the internal parameters of the HMM: Initial State Probabilities, Transition Probabilities, Emission Probabilities
6. Build POS Transition Probabilities
7. Build Emission Probabilities
8. Define Prediction Function
9. Test the Model
10. Verify POS prediction on certain words.

Program:

```
import nltk
from nltk.corpus import brown
from nltk.tag.hmm import HiddenMarkovModelTrainer
import string

nltk.download('brown')
nltk.download('universal_tagset')

sentences = brown.sents()

tokens = []
for sent in sentences:
    for word in sent:
        word = word.lower()
        if word not in string.punctuation:
            tokens.append(word)
```

```
print("==========================================")
print(" HMM POS Tagging using Brown Corpus ")
print("==========================================")
print(f"Total Tokens in Corpus : {len(tokens)}")

tagged_sentences = brown.tagged_sents(tagset='universal')

tagged_sentences = [
    [(word.lower(), tag) for word, tag in sent if word not in string.punctuation]
    for sent in tagged_sentences
]

print(f"Total Tagged Sentences : {len(tagged_sentences)}")

trainer = HiddenMarkovModelTrainer()
hmm_model = trainer.train_supervised(tagged_sentences)

print("\nHMM Model Trained Successfully")

transition_prob = hmm_model._transitions

print("\nSample POS Transition Probabilities:")
for state in list(transition_prob.keys())[:5]:
    print(f"{state} -> {transition_prob[state]}")

emission_prob = hmm_model._outputs

print("\nSample Emission Probabilities:")
for state in list(emission_prob.keys())[:5]:
    print(f"{state} -> {emission_prob[state]._freqdist.most_common(5)}")

def predict_pos(sentence):
    sentence = [word.lower() for word in sentence]
    return hmm_model.tag(sentence)

user_sentence = input("\nEnter a sentence for POS tagging:\n")
user_words = user_sentence.split()

predicted_tags = predict_pos(user_words)

print("\n------------------------------------------")
print(" POS Prediction for Given Sentence ")
print("------------------------------------------")
for word, tag in predicted_tags:
    print(f"{word:<12} -> {tag}")

user_test_words = input(
    "\nEnter words to verify POS (space-separated):\n"
).split()
```

```
print("\n----------------------------------------")
print(" POS Verification on Individual Words ")
print("----------------------------------------")

for word in user_test_words:
    tag = hmm_model.tag([word.lower()])
    print(f"{word:<12} -> {tag[0][1]}")
```

Output:

```
    ========================================
     HMM POS Tagging using Brown Corpus
    ========================================
    Total Tokens in Corpus : 1034378
    Total Tagged Sentences : 57340

    HMM Model Trained Successfully

    Sample POS Transition Probabilities:
    DET -> <MLEProbDist based on 136699 samples>
    NOUN -> <MLEProbDist based on 241376 samples>
    ADJ -> <MLEProbDist based on 80599 samples>
    VERB -> <MLEProbDist based on 176495 samples>
    ADP -> <MLEProbDist based on 144471 samples>

    Sample Emission Probabilities:
    DET -> [('the', 69968), ('a', 23070), ('his', 6957), ('this', 5144), ('an', 3726)]
    NOUN -> [('time', 1597), ('man', 1203), ('af', 995), ('years', 949), ('way', 899)]
    ADJ -> [('other', 1702), ('new', 1635), ('first', 1031), ('many', 996), ('more', 989)]
    VERB -> [('is', 10108), ('was', 9815), ('be', 6374), ('had', 5132), ('are', 4394)]
    ADP -> [('of', 36410), ('in', 20866), ('to', 11158), ('for', 9481), ('with', 7286)]

    Enter a sentence for POS tagging:
    the world
    /usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:335: RuntimeWarning: overflow encountered in cast
      O[i, k] = self._output_logprob(si, self._symbols[k])

    ----------------------------------------
     POS Prediction for Given Sentence
    ----------------------------------------
    the          -> DET
    world        -> NOUN

    Enter words to verify POS (space-separated):
    beautiful

    ----------------------------------------
     POS Verification on Individual Words
    ----------------------------------------
    beautiful    -> ADJ
```

Learning Outcome:

Upon completion of this experiment, HMM based model was built to predict the next word with POS-based suggestions, and evaluated the effectiveness of context-driven predictive text generation using the Brown Corpus.